Bank Management System

Class Documentation — Java 17 · Swing · MySQL (JDBC)

Project: Bank Management System | Launcher: run.bat | Entry point: App

1. Project Overview

A Windows desktop banking application built with plain Java 17, Swing for the GUI and the MySQL Connector/J driver for persistence. There is no framework; the code is split into three packages that form a simple layered architecture:

- **classes** – data models (plain data holders).
- **db** – database access (connections + all SQL).
- **ui** – Swing screens.
- **(default)** – `App`, the program entry point.

Data flows one way: a user action in the **ui** calls the **db** layer, which loads/stores **classes** model objects, which the ui then displays.

Class	Package	Purpose (one line)
App	(default)	Starts the app; sets look-and-feel; opens the login window.
User	classes	Abstract base account: id, username, password, role.
Admin	classes	An admin login (extends User) with an admin id.
Customer	classes	A customer login (extends User) with profile details.
Account	classes	A bank account: number, balance, type, status, open date.
Transaction	classes	One ledger entry: deposit / withdrawal / transfer.
CustomerAccountDTO	classes	Immutable record joining a customer with its account.
DBConfig	db	Connections + credentials + auto-creates the database.
DatabaseOperations	db	The DAO: every SQL statement lives here.
LoginFrame	ui	Login screen (role + username + password).
AdminDashboard	ui	Admin control panel (register / directory / audit log).
CustomerDashboard	ui	Customer portal (balance / deposit / withdraw / transfer).

2. Default package — App

App.java entry point

Program start-up. Applies the Nimbus look-and-feel (falls back to the system look-and-feel), then creates and shows `LoginFrame` on the Swing Event Dispatch Thread (EDT).

Member	What it does
`main(String[] args)`	Loops the installed look-and-feels, selects Nimbus, then calls `SwingUtilities.invokeLater(...)` to show `LoginFrame`.

3. classes package — data models

Plain data-holder classes (POJOs). They carry data between the **db** and **ui** layers. No SQL and no business logic lives here.

User.java `abstract base`

Every logged-in account is a `User`. `Admin` and `Customer` extend it.

Fields

- `int userId` – DB `users.user_id`
- `String username`
- `String password`
- `String role` – "Admin" or "Customer"

Constructors

Constructor	Used by
`User()`	Admin()/Customer() no-arg constructors; DAO creates `new Admin()` / `new Customer()`.
`User(int userId, String, String, String role)`	Admin(int,...)/Customer(int,...) via `super(...)`.
`User(String, String, String role)`	Admin/Customer (String,...) constructors via `super(...)`.

Methods (functions)

Method	What it does	Where it's used
`getUserId()`	Returns userId	AdminDashboard header, toString()
`setUserId(int)`	Sets userId	DatabaseOperations.validateLogin
`getUsername()`	Returns username	LoginFrame welcome, AdminDashboard header
`setUsername(String)`	Sets username	validateLogin
`getPassword()`	Returns password	Standard accessor
`setPassword(String)`	Sets password	validateLogin
`setRole(String)`	Sets role	Admin()/Customer() constructors stamp the role
`toString()`	Debug string	Logging/debug

Note: `role` has only a setter; subclasses stamp it, and the DAO branches on the role string rather than a getter.

Admin.java extends User

Represents an administrator session.

Fields

- `int adminId`

Constructors

- `Admin()` – calls `super()` and `setRole("Admin")`.
- `Admin(int userId, String username, String password, int adminId)`.
- `Admin(String username, String password, int adminId)`.

Methods

Method	What it does	Where it's used
`setAdminId(int)`	Sets adminId	DatabaseOperations.validateLogin
`toString()`	Debug string	Debug

Customer.java extends User

Represents a customer session and profile.

Fields

- `int customerId`
- `String name`, `String email`, `String phone`, `String address`

Constructors

- `Customer()` – `super()` + `setRole("Customer")`.
- `Customer(int userId, String username, String password, int customerId, String name, String email, String phone, String address)`.
- `Customer(String username, String password, String name, String email, String phone, String address)` – used when an admin registers a customer.

Methods

Method	What it does	Where it's used
`getCustomerId()` / `setCustomerId(int)`	Get/set customerId	validateLogin, CustomerDashboard
`getName()` / `setName(String)`	Get/set name	validateLogin, CustomerDashboard header
`getEmail()` / `setEmail(String)`	Get/set email	validateLogin, dashboard header
`getPhone()` / `setPhone(String)`	Get/set phone	validateLogin, admin directory
`getAddress()` / `setAddress(String)`	Get/set address	validateLogin, createCustomerWithAccount
`toString()`	Debug string	Debug

Account.java standalone

A bank account.

Fields

- `long accountNumber`, `int customerId`, `double balance`
- `String accountType` ("Savings"/"Current"), `String status` ("Active"/"Closed")
- `Date openDate`

Constructors

- `Account()` – empty.
- `Account(long, int, double, String accountType, String status, Date openDate)` – full.
- `Account(long, int, double, String accountType)` – defaults status "Active" and openDate now; used by AdminDashboard when creating.

Methods

Method	What it does	Where it's used
`getAccountNumber()` / `setAccountNumber(long)`	Get/set account number	DAO (all money ops), CustomerDashboard
`getCustomerId()` / `setCustomerId(int)`	Get/set owner	createCustomerWithAccount
`getBalance()` / `setBalance(double)`	Get/set balance	DAO, balance card
`getAccountType()` / `setAccountType(String)`	Get/set type	createCustomerWithAccount, summary card
`getStatus()` / `setStatus(String)`	Get/set status	deposit/withdraw/transfer guards
`setOpenDate(Date)`	Sets open date	getAccountDetails / getAccountByCustomerId
`toString()`	Debug string	Debug

Transaction.java standalone

A single ledger entry (audit record).

Fields

- `int transactionId`, `long accountNumber`, `String type` ("DEPOSIT"/"WITHDRAWAL"/"TRANSFER"), `double amount`, `Timestamp timestamp`, `String description`

Constructors

- `Transaction()` – empty.
- `Transaction(int, long, String, double, Timestamp, String)` – full.
- `Transaction(long, String, double, String)` – sets timestamp = now.

Methods

Method	What it does	Where it's used
`getTransactionId()` / `setTransactionId(int)`	Get/set id	History & audit tables
`getAccountNumber()` / `setAccountNumber(long)`	Get/set account	DAO ledger reads
`getType()` / `setType(String)`	Get/set type	History & audit tables
`getAmount()` / `setAmount(double)`	Get/set amount	History & audit tables
`getTimestamp()` / `setTimestamp(Timestamp)`	Get/set time	History & audit tables
`getDescription()` / `setDescription(String)`	Get/set description	History & audit tables
`toString()`	Debug string	Debug

CustomerAccountDTO.java record

An immutable Java `record` that joins a customer with its account, used only to fill the admin "Customer Directory" table.

Components (accessors)

`customerId()`, `userId()`, `name()`, `email()`, `phone()`, `address()`, `accountNumber()`, `balance()`, `accountType()`, `status()`.

Read in `AdminDashboard.loadCustomerData()` to build each table row. Because it is a record, Java generates the constructor, accessors, `equals`, `hashCode` and `toString` automatically — no getters are written by hand.

4. db package — database layer

DBConfig.java `singleton`

Owns the JDBC connection settings and hands out connections. It also **auto-creates** the database, tables and seed data on first run, so the app works on any machine with MySQL.

Fields

- `static volatile DBConfig instance` – lazy singleton.
- `String url`, `String username`, `String password`.

Methods

Method	What it does
`DBConfig()` (private)	Resolves URL/user/password, then loads & registers the MySQL driver (`Class.forName` + `DriverManager.registerDriver`).
`resolve(sysProp, envVar, default)`	Precedence: system property → environment variable → built-in default.
`getInstance()`	Double-checked-locking singleton accessor.
`getConnection()`	Returns a live connection; if the DB is missing (error 1049) it calls `createDatabaseAndSchema()` and retries.
`createDatabaseAndSchema()`	Parses the URL, runs `CREATE DATABASE IF NOT EXISTS`, then `initializeTables()`.
`initializeTables(stmt)`	Creates `users`, `customers`, `accounts`, `transactions`, and seeds demo data if `users` is empty.
`extractDatabaseName(url)`	Pulls `bank_db` out of the JDBC URL.
`extractBaseUrl(url)`	Builds the server-only URL (no database) for the initial connection.
`closeConnection(conn)` `static`	Null-safe close of a connection.
`closeResources(rs, stmt, conn)` `static`	Null-safe close of ResultSet + Statement + Connection.

> Credential defaults: URL `jdbc:mysql://localhost:3306/bank_db`, user `root`, password `paswd`. Override with `-Ddb.password=...` or the `DB_PASSWORD` environment variable.

DatabaseOperations.java DAO

The single place that runs SQL. Every method opens a connection from `DBConfig`, uses a `PreparedStatement` (safe against SQL injection) and closes resources in a `finally` block. Money operations use transactions (`setAutoCommit(false)` + `commit` / `rollback`).

Field

- `DBConfig dbConfig` – the singleton, obtained in the constructor.

Methods

Method	What it does
`validateLogin(username, password, role)`	SELECTs the user (LEFT JOIN customers). Returns an `Admin`, a `Customer`, or `null` if credentials/role don't match.
`createCustomerWithAccount(Customer, Account)`	One atomic transaction: inserts user → customer → account, then an initial-deposit ledger row if balance > 0. Rolls back on any failure.
`getAccountDetails(accountNumber)`	Returns the `Account` for a number (or null).
`getAccountByCustomerId(customerId)`	Returns the first Active `Account` for a customer (or null).
`deposit(accountNumber, amount)`	Validates amount, adds to balance and logs a DEPOSIT row — atomically.
`withdraw(accountNumber, amount)`	Checks sufficient balance/status, subtracts and logs a WITHDRAWAL row — atomically.
`transferFunds(source, target, amount)`	Atomically moves money between two Active accounts and writes a TRANSFER row for each. Executes updates ordered by account number to avoid deadlocks.
`getTransactionHistory(accountNumber)`	Returns that account's transactions, newest first.
`getAllCustomersWithAccounts()`	Returns `List<CustomerAccountDTO>` for the admin directory.
`getAllTransactions()`	Returns every transaction for the admin audit log.

5. ui package — Swing screens

LoginFrame.java `extends JFrame`

The login window: a Role drop-down (Customer/Admin), Username, Password, Login and Exit buttons, and a footer with the demo credentials.

Fields

- `JTextField usernameField` , `JPasswordField passwordField` , `JComboBox<String> roleComboBox`
- `JButton loginButton` , `JButton exitButton`
- `DatabaseOperations dao`

Methods

Method	What it does
`LoginFrame()`	Creates the DAO and builds the UI.
`initUI()`	Builds the header/form/footer, wires Login, Exit (System.exit) and Enter-to-submit.
`handleLogin()`	Validates non-empty input, calls `dao.validateLogin` ; on success opens `AdminDashboard` or `CustomerDashboard` , otherwise shows an error. Catches `SQLException` with a DB-connection message.

AdminDashboard.java `extends JFrame`

The admin control panel with three tabs and a header showing the logged-in admin plus a Logout button.

Fields

- `Admin admin`, `DatabaseOperations dao`
- Registration form: `usernameField, passwordField, nameField, emailField, phoneField, addressArea, accountNumberField, initialDepositField, accountTypeCombo`
- Tables: `customerTable/customerTableModel`, `auditTable/auditTableModel`

Methods

Method	What it does
`AdminDashboard(Admin)`	Builds the UI, then loads customer + audit data.
`initUI()`	Header + tabbed pane (3 tabs) + change-listener that refreshes data when a tab is opened.
`createCustomerRegistrationPanel()`	Builds the registration form (rows for credentials, profile, account type/number/deposit, address) and the Register button.
`createCustomerDirectoryPanel()`	Builds the read-only directory table + Refresh button.
`createAuditLogPanel()`	Builds the read-only audit table + Refresh button.
`handleRegisterCustomer()`	Validates all fields (lengths, email, positive account number, non-negative deposit), builds a `Customer` + `Account`, calls `dao.createCustomerWithAccount`, then refreshes tables and clears the form.
`loadCustomerData()`	Fills the directory table from `dao.getAllCustomersWithAccounts()`.
`loadAuditData()`	Fills the audit table from `dao.getAllTransactions()`.
`clearRegistrationForm()`	Resets all inputs and generates a fresh account number.
`generateAccountNumber()`	Random 10-digit account number via `ThreadLocalRandom`.

CustomerDashboard.java `extends JFrame`

The customer portal: a blue balance card, Deposit / Withdraw / Transfer buttons, and a live transaction-history table. Header shows the customer name + email and a Logout button.

Fields

- `Customer customer`, `DatabaseOperations dao`, `Account customerAccount`
- `JLabel balanceValueLabel`, `JLabel accountInfoLabel`
- `JTable transactionTable`, `DefaultTableModel tableModel`

Methods

Method	What it does
`CustomerDashboard(Customer)`	Loads the customer's account, builds the UI, then loads history.
`loadCustomerAccount()`	Fetches the account via `dao.getAccountByCustomerId`.
`initUI()`	Builds header, balance card, action buttons and the history table.
`createActionButton(text, color)`	Helper that styles the three action buttons.
`handleDeposit()`	Prompts for an amount, validates it, calls `dao.deposit`, then refreshes.
`handleWithdraw()`	Prompts, validates, checks balance, calls `dao.withdraw`, then refreshes.
`handleTransfer()`	Dialog for target account + amount, validates (can't self-transfer, positive, sufficient funds), calls `dao.transferFunds`, then refreshes.
`refreshAccountState()`	Re-reads the account and updates the balance card + history table.
`loadTransactionHistory()`	Fills the table from `dao.getTransactionHistory`.

6. Database schema (bank_db)

Table	Key columns	Relationship
users	user_id (PK), username (unique), password, role	root
customers	customer_id (PK), user_id (FK), name, email, phone, address	→ users (1:1, cascade delete)
accounts	account_number (PK), customer_id (FK), balance, account_type, status, open_date	→ customers (1:N)
transactions	transaction_id (PK), account_number (FK), type, amount, timestamp, description	→ accounts (N:1)

CHECK constraints enforce role (Admin/Customer), account_type (Savings/Current), status (Active/Closed) and transaction type (DEPOSIT/WITHDRAWAL/TRANSFER). The same definitions exist in `DBConfig.initializeTables()` (auto-run) and `schema.sql` (manual).

7. Seed / demo logins

Role	Username	Password	Account	Balance
Admin	admin	admin123	—	—

Customer	san	user123	1000000001 (Savings)	$5,000.00
Customer	jane_smith	user123	1000000002 (Current)	$2,500.50

8. Execution flow

`App` → `LoginFrame` → `DatabaseOperations.validateLogin` → `AdminDashboard` **or** `CustomerDashboard`. Every button calls a `DatabaseOperations` method, which talks to MySQL through `DBConfig` and returns **classes** objects for the ui to display.

9. Build & run

- Double-click `run.bat`, or run:
  `javac -cp "lib\mysql-connector-j-8.3.0.jar" -d bin -sourcepath src src\App.java`
  `java -cp "bin;lib\mysql-connector-j-8.3.0.jar" App`
- The MySQL driver `lib/mysql-connector-j-8.3.0.jar` must stay on the classpath.
- The database and seed data are created automatically on first run.